Programowanie obiektowe, kolokwium 1, 16.05.2025

Podczas kolokwium wszystkie działania wykonywane na ekranie komputera są rejestrowane i mogą zostać odtworzone przez prowadzących. Po napisaniu kolokwium, katalog z projektem należy spakować do formatu zip lub tar.gz i wysłać go przy użyciu kampusu. Proszę nie wyłączać komputera. Podczas kolokwium można korzystać ze wszystkich zasobów internetu, w tym z samodzielnie przygotowanych materiałów z wyłączeniem stron, aplikacji i pluginów opartych o sztuczną inteligencję. Podczas kolokwium zabroniona jest wzajemna komunikacja oraz publikowanie i dostęp do kodu stworzonego na potrzeby kolokwium. Zabronione jest korzystanie z innych urządzeń niż udostępniony komputer. Przedstawione poniżej kroki stanowią propozycję kolejności rozwiązywania zadania. Po przeczytaniu całości można zdecydować o rozwiązywaniu w innej kolejności.

Za niecałe dwa dni w Polsce rozpocznie się pierwsza tura wyborów prezydenckich. Po zakończeniu głosowania wyniki z każdej komisji wyborczej są przekazywane do Państwowej Komisji Wyborczej, która po ich podliczeniu, publikuje je w formie arkuszy danych. Wyniki te są przedstawiane z podziałem na gminy, powiaty i województwa. Na podstawie analogicznych danych z poprzednich wyborów prezydenckich przygotowano pliki z wynikami. Napisz aplikację, która umożliwi analizę i wizualizację tych danych.

Krok 1. ☆☆☆
Utwórz rekord Candidate zawierający nazwę kandydata.

Krok 2. ★☆☆
Utwórz klasę Election, w której zdefiniujesz prywatne pole candidates - listę obiektów Candidate. Dodaj do niej metodę, która zwróci nową listę będącą kopią oryginalnej listy candidates, zachowując jednak te same referencje do obiektów.

Zapoznaj się z plikiem kandydaci.txt, w którym w kolejnych wierszach wymienieni są kandydaci w wyborach prezydenckich 2020 roku.

Krok 3. ★★☆
W klasie Election dodaj metodę populateCandidates(), która przyjmie jako argument ścieżkę do pliku z kandydatami i wczyta z niego dane, zapisując je do listy candidates. Zaimplementuj także metodę populate(), która wywoła populateCandidates() i będzie dalej rozbudowywana w kolejnych krokach.

Krok 4. ★☆☆

Utwórz klasę ElectionTurn, której konstruktor przyjmie listę obiektów Candidate i zapisze ją w polu klasy. Następnie w klasie Election dodaj pola firstTurn i secondTurn typu ElectionTurn. Obiekt pierwszej tury wyborów należy stworzyć od razu, obiekt drugiej początkowo powinien być nullem (opis jego stworzenia jest w kroku 15). Napisz akcesory do tych obiektów.

Przeanalizuj plik 1.csv, który zawiera wyniki pierwszej tury głosowania w poszczególnych gminach. Jego pierwsze trzy kolumny opisują lokalizację głosowania, a kolejne - liczbę głosów zdobytych przez kolejnych kandydatów. Kolejność kandydatów odpowiada kolejności kolumn w pliku kandydaci.txt. Plik 2.csv dotyczy drugiej tury i ma analogiczną strukturę.

Krok 5. ★☆☆
Utwórz klasę Vote, która zawiera:

- Prywatną mapę votesForCandidate, w której kluczem jest obiekt Candidate, a wartością - liczba uzyskanych przez niego głosów.
- Prywatną listę location, przechowującą w kolejności: nazwę województwa, powiatu i gminy.

Krok 6. ★★☆
W klasie Vote dodaj publiczną, statyczną metodę fromCsvLine(), która:

- Przyjmuje wiersz tekstu z pliku CSV (1.csv lub 2.csv).
- Parsuje zawartość tego wiersza i na jej podstawie tworzy nowy obiekt Vote.
- Wypełnia w tym obiekcie obie struktury danych utworzone wcześniej w kroku 5. (tj. mapę głosów votesForCandidate oraz listę location).

Krok 7. ★★☆
W klasie ElectionTurn:

- Dodaj prywatne pole votes, będące listą obiektów Vote.
- Zaimplementuj w niej także metodę populate(), która otrzyma ścieżkę do pliku z wynikami głosowań. przyjmującą ścieżkę do pliku z wynikami głosowań. Metoda powinna wczytać dane z pliku i zapełnić nimi listę votes.

Następnie w klasie Election, w metodzie populate() (utworzonej w kroku 3), wywołaj firstTurn.populate(), przekazując do niej ścieżkę do pliku 1.csv.

Krok 8. ★★☆
W klasie Vote dodaj metodę summarize(), która:

- Przyjmie jako argument listę obiektów Vote - wyniki głosowania z poszczególnych gmin.
- Utworzy i zwróci nowy obiekt Vote, którego:
  - Mapa votesForCandidate będzie zawierać te same klucze co mapy w obiektach przekazanych w argumencie.
  - Wartości przy tych kluczach będą sumami głosów oddanych na poszczególnych kandydatów we wszystkich obiektach z listy.
- Ustawi pole location nowego obiektu jako pustą listę.

Krok 9. ★☆☆
W klasie Vote dodaj metody, obie przejmujące kandydata jako argument:

- votes(), która zwraca liczbę głosów zapisanych w obiekcie klasy Vote,
- percentage(), która zwraca procentowy udział głosów oddanych na niego względem łącznej liczby głosów zapisanych w obiekcie Vote.

Krok 10. ★☆☆
W klasie Vote nadpisz metodę toString(), tak aby zwracała napis zawierający w kolejnych liniach imiona i nazwiska kandydatów wraz z ich procentowymi wynikami. Kolejność może być dowolna. Następnie przetestuj wywołanie tej metody w metodzie main().

Krok 11. ★★☆
Wyświetlenie napisu z poprzedniego kroku wymaga wielokrotnego wykorzystania sumy wszystkich głosów. Zagwarantuj, że suma ta będzie obliczana tylko raz.

Krok 12. ★★☆

W klasie ElectionTurn zaimplementuj metodę winner(), która zwraca kandydata będącego zwycięzcą tury. Zwycięzcą jest kandydat, który zdobył ponad 50% wszystkich oddanych głosów. Jeśli żaden kandydat nie osiągnie tego progu, metoda powinna rzucić (samodzielnie napisany) wyjątek NoWinnerException.

Krok 13. ★☆☆
W klasie Election dodaj prywatne pole winner, które będzie przechowywać referencję do zwycięskiego kandydata. Stwórz akcesor do tego pola. W metodzie populate() klasy Election wywołaj metodę winner() na obiekcie firstTurn. Jeśli nie zostanie rzucony wyjątek NoWinnerException, ustaw wartość zwróconą przez tę metodę jako wartość pola winner w klasie Election.

Krok 14. ★★☆
W klasie ElectionTurn zaimplementuj metodę runoffCandidates(), która zwróci listę dwóch kandydatów z największą liczbą głosów.

Krok 15. ★☆☆
Jeśli wywołanie metody winner() na obiekcie firstTurn spowoduje rzucenie wyjątku, należy:

- Uzyskać listę kandydatów, którzy zakwalifikowali się do drugiej tury,
- Utworzyć obiekt secondTurn,
- Wczytać dane z pliku 2.csv i zapełnić nimi obiekt secondTurn,
- Na końcu metody populate() odczytać zwycięzcę drugiej tury i ustawić go jako zwycięzcę całych wyborów.

Krok 16. ★★★
W klasie Vote zaimplementuj publiczną, statyczną metodę filterByLocation(), która przyjmuje dwie listy jako argumenty:

- listę obiektów klasy Vote,
- listę napisów opisujących lokalizację.

Lista lokalizacji może zawierać:

- jeden element - województwo (np. "lubelskie") - zwróci wszystkie obiekty z województwa,
- dwa elementy - województwo i powiat (np. "lubelskie", "Lublin") - zwróci wszystkie obiekty z powiatu,
- trzy elementy - województwo, powiat i gmina (np. "lubelskie", "Lublin", "m. Lublin") - zwróci dokładnie jeden element odpowiadający gminie.

Metoda powinna zwracać nową listę obiektów Vote, których lokalizacja odpowiada wskazanemu filtrowi.

Krok 17. ★☆☆

W klasie Vote zmodyfikuj metodę summarize(), tak aby przyjmowała jako argument listę lokalizacji (location) i ustawia ła ją w nowym obiekcie Vote.

W klasie ElectionTurn zaimplementuj przeciążone metody summarize() wykorzystując statyczny polimorfizm:

- wersja bezargumentowa powinna podsumowywać wszystkie głosy w danej turze,
- wersja z argumentem w postaci listy nazw lokalizacji (location) powinna najpierw przefiltrować głosy, a dopiero potem wykonać podsumowanie.

Przetestuj działanie obu wersji metody summarize() w funkcji main().

Krok 18. ☆☆☆
Zapoznaj się z klasą VoivodeshipMap, która zawiera statyczną mapę ścieżek służących do rysowania symbolicznych kształtów polskich województw. Metoda saveToSvg() tej klasy generuje plik SVG z mapą Polski. Przetestuj działanie tej metody, aby upewnić się, że mapa jest poprawnie generowana.

Krok 19. ★★☆
W klasie VoivodeshipMap zaimplementuj metodę, która zwraca listę nazw polskich województw. Następnie, w metodzie main(), wykorzystaj tę metodę wraz z metodą summarize() z klasy ElectionTurn, aby wygenerować podsumowanie wyników drugiej tury wyborów dla każdego województwa. Dobierz odpowiednią strukturę danych do przechowywania tych wyników według własnego uznania.

Krok 20. ★★☆

Zmodyfikuj klasę VoivodeshipMap, tak aby kolor województwa był określany przez chronioną metodę, która przyjmuje nazwę województwa jako argument.

Następnie utwórz klasę SelectableMap jako podklasę VoivodeshipMap. Zaimplementuj w niej publiczną metodę select(), która przyjmuje nazwę województwa jako argument. Na obrazie wygenerowanym za pomocą metody saveToSvg() ostatnio zaznaczone województwo powinno być wyróżnione kolorem innym od pozostałych. Działanie przetestuj w metodzie main().

Krok 21. ★★★
Utwórz klasę VoteMap jako klasę dziedziczącą po VoivodeshipMap. Zaprojektuj ją w taki sposób, aby w jej obiekcie można było zapisać wyniki głosowania uzyskane w kroku 19. Należy również dowolnie powiązać kandydatów z wybranymi kolorami. Wynikowy obraz, generowany przez metodę saveToSvg(), powinien przedstawiać każde województwo w kolorze odpowiadającym kandydatowi, który zdobył tam największą liczbę głosów. Działanie przetestuj w metodzie main().